

SIMATS ENGINEERING ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1703

COURSE OUTCOME COVERED

CO3: Analyze knowledge representation and reasoning using propositional logic, FOL, inference, resolution, chaining methods, knowledge representation to perform logical reasoning for drawing valid conclusions and decision making. (BL4)

Assessment Tool Weightage: 20%

QUESTIONS: 5

Time Duration: 1 Hour
Total Marks:30

Annexure A

Resolution Algorithm Implementation

Code of Conduct:

I Hashini S / 192524035 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Hashini S

Verification Program

To verify each hand-worked proof, a general-purpose PL-Resolution engine was implemented in Python. The program represents each clause as a set of literals, repeatedly resolves pairs of clauses that contain complementary literals, and reports the Empty Clause ($\square$) whenever a contradiction is found. The same script (resolution.py) was used to solve all five questions above — its source code and full console output are shown below.

Source Code

```
resolution.py (1/2)

1  """
2  Propositional Logic Resolution Algorithm
3  -----
4  Generic PL-Resolution engine + application to 5 knowledge bases
5  (Rain/Wet, Assignment/Marks, Library/Borrow, Aptitude/Placement,
6   Password/Authentication/Access).
7
8  A clause is represented as a frozenset of literal strings.
9  A negative literal is written with a leading '~' (e.g. "~Rain").
10 """
11
12 def neg(lit):
13     """Return the complement of a literal."""
14     return lit[1:] if lit.startswith('~') else '~' + lit
15
16
17 def resolve(ci, cj):
18     """Return the set of all resolvents obtainable from clauses ci and cj."""
19     resolvents = set()
20     for li in ci:
21         if neg(li) in cj:
22             new_clause = (ci - {li}) | (cj - {neg(li)})
23             resolvents.add(frozenset(new_clause))
24     return resolvents
25
26
27 def clause_str(c):
28     if len(c) == 0:
29         return " $\square$  (Empty Clause)"
30     return "(" + " v ".join(sorted(c)) + ")"
31
32
33 def pl_resolution(kb_clauses, goal_literal, title):
34     """
35     kb_clauses : list[frozenset] -> CNF clauses of the knowledge base
36     goal_literal : str           -> literal to be proved (e.g. "Wet")
37     """
38     print("=" * 70)
39     print(title)
40     print("=" * 70)
41
42     clauses = set(kb_clauses)
43     neg_goal = frozenset({neg(goal_literal)})
44     clauses.add(neg_goal)
45
46     print("\nCNF Clause Set (KB + Negated Goal):")
47     for i, c in enumerate(clauses, 1):
48         print(f"  C{i}: {clause_str(c)}")
49
50     step = 0
51     seen_pairs = set()
52     all_clauses = set(clauses)
53
54     while True:
55         new_clauses = set()
56         clause_list = list(all_clauses)
57         proved = False
58
59         for i in range(len(clause_list)):
60             for j in range(i + 1, len(clause_list)):
61                 ci, cj = clause_list[i], clause_list[j]
62                 pair_key = frozenset({ci, cj})
```

```

1         if pair_key in seen_pairs:
2             continue
3         seen_pairs.add(pair_key)
4
5         resolvents = resolve(ci, cj)
6         for r in resolvents:
7             step += 1
8             print(f"\nStep {step}: Resolve {clause_str(ci)} with {clause_str(cj)}")
9             print(f"         -> Resolvent: {clause_str(r)}")
10            if len(r) == 0:
11                print("\n*** Empty Clause (□) derived – CONTRADICTION FOUND ****")
12                proved = True
13                break
14            new_clauses.add(r)
15        if proved:
16            break
17    if proved:
18        break
19
20    if proved:
21        print(f"\nCONCLUSION: Goal '{goal_literal}' is PROVED TRUE by refutation.\n")
22        return True
23
24    if new_clauses.issubset(all_clauses):
25        print("\nNo new clauses can be generated. Goal CANNOT be proved.\n")
26        return False
27
28    all_clauses |= new_clauses
29
30
31    # -----
32    # QUESTION 1: Rain and Wet Ground
33    # KB: Rain -> Wet , Rain
34    # CNF: (~Rain v Wet) , (Rain)
35    # -----
36    kb1 = [frozenset({"~Rain", "Wet"}), frozenset({"Rain"})]
37    pl_resolution(kb1, "Wet", "QUESTION 1: Rain and Wet Ground")
38
39
40    # -----
41    # QUESTION 2: Student Assignment Submission
42    # KB: Submit -> Marks , Submit
43    # -----
44    kb2 = [frozenset({"~Submit", "Marks"}), frozenset({"Submit"})]
45    pl_resolution(kb2, "Marks", "QUESTION 2: Student Assignment Submission (Rahul)")
46
47
48    # -----
49    # QUESTION 3: Library Membership
50    # KB: Member -> Borrow , Member
51    # -----
52    kb3 = [frozenset({"~Member", "Borrow"}), frozenset({"Member"})]
53    pl_resolution(kb3, "Borrow", "QUESTION 3: Library Membership (Priya)")
54
55
56    # -----
57    # QUESTION 4: Placement Eligibility
58    # KB: Aptitude -> Eligible , Aptitude
59    # -----
60    kb4 = [frozenset({"~Aptitude", "Eligible"}), frozenset({"Aptitude"})]
61    pl_resolution(kb4, "Eligible", "QUESTION 4: Placement Eligibility (Arun)")
62
63
64    # -----
65    # QUESTION 5: Access Control System
66    # KB: Password -> Auth , Auth -> Access , Password
67    # -----
68    kb5 = [frozenset({"~Password", "Auth"}),
69          frozenset({"~Auth", "Access"}),
70          frozenset({"Password"})]
71    pl_resolution(kb5, "Access", "QUESTION 5: Access Control System")

```

Question 1 — Rain and Wet Ground

Knowledge Base

- If it rains, then the ground becomes wet.
- It is raining.

Task 1: Propositional Logic Representation

Let Rain = "It is raining"
Let Wet = "The ground is wet"

Premise 1: Rain $\rightarrow$ Wet
Premise 2: Rain
Goal: Wet

Task 2: Conversion to CNF

An implication $P \rightarrow Q$ is logically equivalent to $(\neg P \vee Q)$. Applying this equivalence to every conditional statement gives the CNF clauses below:

Clause	Propositional Statement	CNF Form
C1	Rain $\rightarrow$ Wet	$(\neg \text{Rain} \vee \text{Wet})$
C2	Rain	(Rain)

Negating the Goal

Goal: Wet (the ground is wet)

Negated Goal (added to the clause set): $\neg \text{Wet} \rightarrow$ clause: $(\neg \text{Wet})$

Task: Applying the Resolution Algorithm

Initial clause set (Knowledge Base CNF + Negated Goal):

C1: $(\neg \text{Rain} \vee \text{Wet})$
C2: (Rain)
C3: $(\neg \text{Wet})$ <- negated goal

Step	Clause 1	Clause 2	Resolvent
1	(Rain)	$(\neg \text{Rain} \vee \text{Wet})$	(Wet)
2	(Wet)	$(\neg \text{Wet})$	$\square$ (Empty Clause)

Empty Clause and Conclusion

Resolving the final two complementary unit clauses produces the Empty Clause ($\square$), a contradiction. Since assuming $\neg(\text{Wet})$ leads to a contradiction with the Knowledge Base, the original goal "Wet" must be TRUE.

✓ Goal Proved: Wet is TRUE (Empty Clause $\square$ derived).

Program Output (Screenshot)

The resolution steps above were verified by running a Python implementation of the PL-Resolution algorithm. The screenshot below shows the actual program output for this question:

```
Terminal - Question 1 Output

QUESTION 1: Rain and Wet Ground

CNF Clause Set (KB + Negated Goal):
  C1: (Rain)
  C2: (~Wet)
  C3: (Wet v ~Rain)

Step 1: Resolve (Rain) with (Wet v ~Rain)
      -> Resolvent: (Wet)

Step 2: Resolve (~Wet) with (Wet v ~Rain)
      -> Resolvent: (~Rain)

Step 3: Resolve (~Rain) with (Rain)
      -> Resolvent:  $\square$  (Empty Clause)

*** Empty Clause ( $\square$ ) derived - CONTRADICTION FOUND ***

CONCLUSION: Goal 'Wet' is PROVED TRUE by refutation.
```

Question 2 — Student Assignment Submission

Knowledge Base

- If a student submits the assignment, then the student receives internal marks.
- Rahul submitted the assignment.

Task 1: Propositional Logic Representation

```
Let Submit = "Rahul submits the assignment"
Let Marks  = "Rahul receives internal marks"
```

Premise 1: Submit $\rightarrow$ Marks

Premise 2: Submit

Goal: Marks

Task 2: Conversion to CNF

An implication $P \rightarrow Q$ is logically equivalent to $(\neg P \vee Q)$. Applying this equivalence to every conditional statement gives the CNF clauses below:

Clause	Propositional Statement	CNF Form
C1	Submit $\rightarrow$ Marks	$(\neg \text{Submit} \vee \text{Marks})$
C2	Submit	(Submit)

Negating the Goal

Goal: Marks (Rahul receives internal marks)

Negated Goal (added to the clause set): $\neg \text{Marks} \rightarrow \text{clause: } (\neg \text{Marks})$

Task: Applying the Resolution Algorithm

Initial clause set (Knowledge Base CNF + Negated Goal):

C1: $(\neg \text{Submit} \vee \text{Marks})$

C2: (Submit)

C3: $(\neg \text{Marks})$ <- negated goal

Step	Clause 1	Clause 2	Resolvent
1	(Submit)	$(\neg \text{Submit} \vee \text{Marks})$	(Marks)
2	(Marks)	$(\neg \text{Marks})$	$\square$ (Empty Clause)

Empty Clause and Conclusion

Resolving the final two complementary unit clauses produces the Empty Clause ($\square$), a contradiction. Since assuming $\neg(\text{Marks})$ leads to a contradiction with the Knowledge Base, the original goal "Marks" must be TRUE.

✓ Goal Proved: Marks is TRUE (Empty Clause $\square$ derived).

Program Output (Screenshot)

The resolution steps above were verified by running a Python implementation of the PL-Resolution algorithm. The screenshot below shows the actual program output for this question:

```
Terminal - Question 2 Output

QUESTION 2: Student Assignment Submission (Rahul)

CNF Clause Set (KB + Negated Goal):
C1: (~Marks)
C2: (Submit)
C3: (Marks v ~Submit)

Step 1: Resolve (~Marks) with (Marks v ~Submit)
-> Resolvent: (~Submit)

Step 2: Resolve (Submit) with (Marks v ~Submit)
-> Resolvent: (Marks)

Step 3: Resolve (~Submit) with (Submit)
-> Resolvent: □ (Empty Clause)

*** Empty Clause (□) derived - CONTRADICTION FOUND ***

CONCLUSION: Goal 'Marks' is PROVED TRUE by refutation.
```

Figure 2.1 — Terminal output proving Marks is TRUE.

Question 3 — Library Membership

Knowledge Base

- If a person is a library member, then the person can borrow books.
- Priya is a library member.

Task 1: Propositional Logic Representation

Let Member = "Priya is a library member"

Let Borrow = "Priya can borrow books"

Premise 1: $\text{Member} \rightarrow \text{Borrow}$

Premise 2: Member

Goal: Borrow

Task 2: Conversion to CNF

An implication $P \rightarrow Q$ is logically equivalent to $(\neg P \vee Q)$. Applying this equivalence to every conditional statement gives the CNF clauses below:

Clause	Propositional Statement	CNF Form
C1	$\text{Member} \rightarrow \text{Borrow}$	$(\neg \text{Member} \vee \text{Borrow})$
C2	Member	(Member)

Negating the Goal

Goal: Borrow (Priya can borrow books)

Negated Goal (added to the clause set): $\neg \text{Borrow} \rightarrow$ clause: $(\neg \text{Borrow})$

Task: Applying the Resolution Algorithm

Initial clause set (Knowledge Base CNF + Negated Goal):

C1: $(\neg \text{Member} \vee \text{Borrow})$

C2: (Member)

C3: $(\neg \text{Borrow})$ <- negated goal

Step	Clause 1	Clause 2	Resolvent
1	(Member)	$(\neg \text{Member} \vee \text{Borrow})$	(Borrow)
2	(Borrow)	$(\neg \text{Borrow})$	$\square$ (Empty Clause)

Empty Clause and Conclusion

Resolving the final two complementary unit clauses produces the Empty Clause ($\square$), a contradiction. Since assuming $\neg(\text{Borrow})$ leads to a contradiction with the Knowledge Base, the original goal "Borrow" must be TRUE.

✓ Goal Proved: Borrow is TRUE (Empty Clause □ derived).

Program Output (Screenshot)

The resolution steps above were verified by running a Python implementation of the PL-Resolution algorithm. The screenshot below shows the actual program output for this question:

```
Terminal - Question 3 Output

QUESTION 3: Library Membership (Priya)

CNF Clause Set (KB + Negated Goal):
  C1: (~Borrow)
  C2: (Member)
  C3: (Borrow v ~Member)

Step 1: Resolve (~Borrow) with (Borrow v ~Member)
      -> Resolvent: (~Member)

Step 2: Resolve (Member) with (Borrow v ~Member)
      -> Resolvent: (Borrow)

Step 3: Resolve (~Borrow) with (Borrow)
      -> Resolvent: □ (Empty Clause)

*** Empty Clause (□) derived - CONTRADICTION FOUND ***

CONCLUSION: Goal 'Borrow' is PROVED TRUE by refutation.
```

Question 4 — Placement Eligibility

Knowledge Base

- If a student clears the aptitude test, then the student is eligible for placement.
- Arun cleared the aptitude test.

Task 1: Propositional Logic Representation

```
Let Aptitude = "Arun clears the aptitude test"
Let Eligible = "Arun is eligible for placement"
```

Premise 1: $\text{Aptitude} \rightarrow \text{Eligible}$

Premise 2: Aptitude

Goal: Eligible

Task 2: Conversion to CNF

An implication $P \rightarrow Q$ is logically equivalent to $(\neg P \vee Q)$. Applying this equivalence to every conditional statement gives the CNF clauses below:

Clause	Propositional Statement	CNF Form
C1	$\text{Aptitude} \rightarrow \text{Eligible}$	$(\neg \text{Aptitude} \vee \text{Eligible})$
C2	Aptitude	(Aptitude)

Negating the Goal

Goal: Eligible (Arun is eligible for placement)

Negated Goal (added to the clause set): $\neg \text{Eligible} \rightarrow \text{clause: } (\neg \text{Eligible})$

Task: Applying the Resolution Algorithm

Initial clause set (Knowledge Base CNF + Negated Goal):

C1: $(\neg \text{Aptitude} \vee \text{Eligible})$

C2: (Aptitude)

C3: $(\neg \text{Eligible})$ <- negated goal

Step	Clause 1	Clause 2	Resolvent
1	(Aptitude)	$(\neg \text{Aptitude} \vee \text{Eligible})$	(Eligible)
2	(Eligible)	$(\neg \text{Eligible})$	$\square$ (Empty Clause)

Empty Clause and Conclusion

Resolving the final two complementary unit clauses produces the Empty Clause ($\square$), a contradiction. Since assuming $\neg(\text{Eligible})$ leads to a contradiction with the Knowledge Base, the original goal "Eligible" must be TRUE.

✓ Goal Proved: Eligible is TRUE (Empty Clause $\square$ derived).

Program Output (Screenshot)

The resolution steps above were verified by running a Python implementation of the PL-Resolution algorithm. The screenshot below shows the actual program output for this question:

```
Terminal - Question 4 Output

QUESTION 4: Placement Eligibility (Arun)

CNF Clause Set (KB + Negated Goal):
C1: (Aptitude)
C2: (¬Eligible)
C3: (Eligible v ¬Aptitude)

Step 1: Resolve (Aptitude) with (Eligible v ¬Aptitude)
-> Resolvent: (Eligible)

Step 2: Resolve (¬Eligible) with (Eligible v ¬Aptitude)
-> Resolvent: (¬Aptitude)

Step 3: Resolve (Aptitude) with (¬Aptitude)
-> Resolvent: □ (Empty Clause)

*** Empty Clause (□) derived - CONTRADICTION FOUND ***

CONCLUSION: Goal 'Eligible' is PROVED TRUE by refutation.
```

Question 5 — Access Control System

Knowledge Base

- If a user enters the correct password, then the user is authenticated.
- If a user is authenticated, then the user is granted access.
- The user entered the correct password.

Task 1: Propositional Logic Representation

Let Password = "The user entered the correct password"

Let Auth = "The user is authenticated"

Let Access = "The user is granted access"

Premise 1: Password $\rightarrow$ Auth

Premise 2: Auth $\rightarrow$ Access

Premise 3: Password

Goal: Access

Task 2: Conversion to CNF

An implication $P \rightarrow Q$ is logically equivalent to $(\neg P \vee Q)$. Applying this equivalence to every conditional statement gives the CNF clauses below:

Clause	Propositional Statement	CNF Form
C1	Password $\rightarrow$ Auth	$(\neg \text{Password} \vee \text{Auth})$
C2	Auth $\rightarrow$ Access	$(\neg \text{Auth} \vee \text{Access})$
C3	Password	(Password)

Negating the Goal

Goal: Access (the user is granted access)

Negated Goal (added to the clause set): $\neg \text{Access} \rightarrow$ clause: $(\neg \text{Access})$

Task: Applying the Resolution Algorithm

Initial clause set (Knowledge Base CNF + Negated Goal):

C1: $(\neg \text{Password} \vee \text{Auth})$

C2: $(\neg \text{Auth} \vee \text{Access})$

C3: (Password)

C4: $(\neg \text{Access})$ <- negated goal

Step	Clause 1	Clause 2	Resolvent
1	(Password)	$(\neg \text{Password} \vee \text{Auth})$	(Auth)
2	(Auth)	$(\neg \text{Auth} \vee \text{Access})$	(Access)

Step	Clause 1	Clause 2	Resolvent
3	(Access)	(¬Access)	□ (Empty Clause)

Empty Clause and Conclusion

Resolving the final two complementary unit clauses produces the Empty Clause (□), a contradiction. Since assuming ¬(Access) leads to a contradiction with the Knowledge Base, the original goal "Access" must be TRUE.

✓ **Goal Proved: Access is TRUE (Empty Clause □ derived).**

Program Output (Screenshot)

The resolution steps above were verified by running a Python implementation of the PL-Resolution algorithm. The screenshot below shows the actual program output for this question:

```

Terminal - Question 5 Output

QUESTION 5: Access Control System

CNF Clause Set (KB + Negated Goal):
C1: (Password)
C2: (Auth v ~Password)
C3: (~Access)
C4: (Access v ~Auth)

Step 1: Resolve (Password) with (Auth v ~Password)
-> Resolvent: (Auth)

Step 2: Resolve (Auth v ~Password) with (Access v ~Auth)
-> Resolvent: (Access v ~Password)

Step 3: Resolve (~Access) with (Access v ~Auth)
-> Resolvent: (~Auth)

Step 4: Resolve (~Access) with (Access v ~Password)
-> Resolvent: (~Password)

Step 5: Resolve (Access v ~Password) with (Password)
-> Resolvent: (Access)

Step 6: Resolve (~Auth) with (Auth v ~Password)
-> Resolvent: (~Password)

Step 7: Resolve (~Auth) with (Auth)
-> Resolvent: □ (Empty Clause)

*** Empty Clause (□) derived - CONTRADICTION FOUND ***

CONCLUSION: Goal 'Access' is PROVED TRUE by refutation.

```